Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3701625.3701673

PDF Download
3701625.3701673.pdf
07 April 2026
Total Citations: 2
Total Downloads: 428

Published: 21 December 2024

Citation in BibTeX format

SBQS 2024: XXIII Brazilian Symposium on Software Quality
November 5 - 8, 2024
Bahia, Salvador, Brazil

RESEARCH-ARTICLE

Code on Demand: A Comparative Analysis of the Efficiency Understandability and Self-Correction Capability of Copilot ChatGPT and Gemini

SAMUEL SILVESTRE BATISTA, Federal University of Piauí, Teresina, PI, Brazil

BRUNO BRANCO, Federal University of Piauí, Teresina, PI, Brazil

OTÁVIO CASTRO, Federal University of Piauí, Teresina, PI, Brazil

GUILHERME AVELINO, Federal University of Piauí, Teresina, PI, Brazil

Open Access Support provided by:

Federal University of Piauí

SBQS '24: Proceedings of the XXIII Brazilian Symposium on Software Quality (November 2024)
https://doi.org/10.1145/3701625.3701673
ISBN: 9798400717772

Code on Demand: A Comparative Analysis of the Efficiency Understandability and Self-Correction Capability of Copilot ChatGPT and Gemini

Samuel Silvestre Batista
Universidade Federal do Piauí
Teresina, Piauí, Brasil
sssb@ufpi.edu.br

Bruno Branco
Universidade Federal do Piauí
Teresina, Piauí, Brasil
brunomendesccb@gmail.com

Otávio Castro
Universidade Federal do Piauí
Teresina, Piauí, Brasil
otaviocury@ufpi.edu.br

Guilherme Avelino
Universidade Federal do Piauí
Teresina, Piauí, Brasil
gaa@ufpi.edu.br

Resumo

The increasing demand for software development and a shortage of skilled developers has catalyzed the emergence of AI-powered code generation tools as potential solutions to enhance productivity and accessibility. While these tools promise to revolutionize software development, limited research critically evaluates their performance, particularly concerning code quality, comprehensibility, and self-correction capabilities. Therefore, this paper investigates the efficacy, comprehensibility, and self-repair capabilities of three prominent AI code generation tools: GitHub Copilot, ChatGPT, and Gemini. We replicate and extend the study by Nguyen and Nadi, evaluating the tools' performance on 33 LeetCode problems across four programming languages: Python, Java, JavaScript, and C. Our findings demonstrate notable variations in performance between the tools and programming languages. Copilot achieved the highest accuracy in Java (87.88%), while ChatGPT consistently performed well across Python, Java, and JavaScript (78.79%). Gemini excelled in JavaScript (75.76%). Copilot generally generated more understandable code, as indicated by lower cognitive and cyclomatic complexity, whereas ChatGPT and Gemini exhibited higher variability. All tools demonstrated promising self-repair capabilities, but their effectiveness varied depending on the type of error and programming language. This study provides valuable insights into the strengths and limitations of these AI-powered tools, informing their practical application in software development.

ACM Reference Format:
Samuel Silvestre Batista, Bruno Branco, Otávio Castro, and Guilherme Avelino. 2024. Code on Demand: A Comparative Analysis of the Efficiency Understandability and Self-Correction Capability of Copilot ChatGPT and Gemini. In *XXIII Brazilian Symposium on Software Quality (SBQS 2024), November 05–08, 2024, Salvador, Brazil.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3701625.3701673

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBQS 2024, November 05–08, 2024, Salvador, Brazil
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1777-2/24/11
https://doi.org/10.1145/3701625.3701673

1 Introdução

A crescente demanda por desenvolvimento de software, impulsionada pela digitalização acelerada em diversos setores, e a escassez de desenvolvedores qualificados criam uma necessidade de soluções que aumentem a produtividade e a acessibilidade no desenvolvimento de software. Ferramentas de geração de código alimentadas por Inteligência Artificial (IA), como Copilot, ChatGPT e Gemini, emergem como potenciais soluções, prometendo revolucionar o desenvolvimento de software. Entretanto, a avaliação dessas ferramentas, particularmente em relação à qualidade, compreensibilidade e capacidade de autocorreção do código gerado, ainda é limitada.

A arquitetura *Transformer* [31] impulsionou o desenvolvimento de modelos generativos notáveis, capazes de gerar texto, traduzir idiomas e produzir código [8, 9, 21, 25]. Porém, a mera capacidade de gerar código sintaticamente correto não garante a produção de software de qualidade, reutilizável e de fácil manutenção [20].

Este artigo apresenta um estudo empírico que investiga a eficácia, a compreensibilidade e a capacidade de autocorreção de três ferramentas populares de geração de código por IA: GitHub Copilot, ChatGPT e Gemini. Replicando e expandindo o estudo de Nguyen e Nadi [18], avaliamos o desempenho das ferramentas em 33 problemas do LeetCode [14] abrangendo quatro linguagens de programação: Python, Java, JavaScript e C. A avaliação se baseia em três métricas principais: (i) *acurácia* — a porcentagem de respostas corretas geradas pela ferramenta; (ii) *compreensibilidade* — medida pela complexidade cognitiva e ciclomática do código; e (iii) *capacidade de autocorreção* — a habilidade da ferramenta em reparar erros no código gerado através de interações (*prompts*) adicionais.

Os resultados demonstram variações significativas no desempenho das ferramentas dependendo da linguagem de programação. O Copilot alcançou a maior acurácia em Java (87,88%), enquanto o ChatGPT apresentou desempenho consistente em Python, Java e JavaScript (78,79%). O Gemini se destacou em JavaScript (75,76%). Em relação à compreensibilidade, o Copilot gerou código com menor complexidade cognitiva de forma mais consistente, enquanto ChatGPT e Gemini apresentaram maior variabilidade. As três ferramentas demonstraram capacidade de autocorreção, mas sua eficácia variou consideravelmente entre os tipos de erro e linguagens.

Este estudo contribui para a área ao fornecer uma análise comparativa aprofundada de três ferramentas proeminentes de geração

de código por IA, destacando seus pontos fortes e limitações. As descobertas fornecem insights valiosos sobre a aplicabilidade prática dessas ferramentas no desenvolvimento de software, orientando desenvolvedores e pesquisadores na tomada de decisão e no direcionamento de pesquisas futuras.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica sobre Inteligência Artificial Generativa e métricas de complexidade de código. A Seção 3 detalha a metodologia empregada no estudo, incluindo a seleção de problemas, coleta de dados e processo de avaliação. A Seção 4 apresenta os resultados da avaliação das ferramentas em relação à acurácia, compreensibilidade e capacidade de autocorreção. A Seção 5 discute os resultados, analisando as implicações práticas e comparando-os com trabalhos relacionados. A Seção 6 aborda as ameaças à validade do estudo e a Seção 7 apresenta trabalhos relacionados. Por fim, a Seção 8 conclui o artigo, resumindo as principais descobertas e delineando direções para pesquisas futuras.

2 Fundamentação Teórica

2.1 Inteligência Artificial Generativa

A Inteligência Artificial Generativa (IAG) abrange modelos de IA projetados para gerar conteúdo novo e coerente a partir de dados fornecidos [11]. O mecanismo de autoatenção, um componente central desses modelos, permite condicionar a computação a todos os vetores ocultos da rede neural. Cada palavra da entrada é codificada em vetores, os quais são analisados conjuntamente para capturar dependências globais dentro dos dados [28].

O modelo *Transformer* [31], uma evolução significativa nessa área, utiliza camadas de *encoders* e *decoders*. Os *encoders* codificam a entrada completamente, enquanto os *decoders* geram a saída com base nessa codificação, considerando apenas as palavras até a posição n-1 para cada palavra na posição n. A cada camada, uma sequência de saída é gerada e propagada para a próxima, com conexões residuais adicionadas para preservar o contexto [28]. Finalmente, uma função *softmax* seleciona a palavra mais adequada ao contexto.

Modelos de Linguagem de Larga Escala (LLMs) têm demonstrado notável capacidade na geração de código a partir de descrições textuais. Um exemplo é o Codex, um modelo GPT treinado em um vasto conjunto de dados de código disponível publicamente no GitHub, o qual apresentou melhorias significativas em relação a modelos anteriores como o GPT-3 [3].

Além da geração de código, os LLMs também se mostram promissores na análise de grandes volumes de código existente, permitindo a extração de padrões, a otimização da refatoração e a facilitação da compreensão de sistemas complexos. Essas capacidades ampliam o papel dos LLMs no desenvolvimento de software, abrangendo desde a escrita de funções autônomas até a documentação automática e o suporte à compreensão de código [3].

2.2 IAs Generativas na Engenharia de Software

A Inteligência Artificial Generativa (GenAI) vem emergindo como uma força transformadora na Engenharia de Software, com potencial para revolucionar diversas etapas do ciclo de vida de desenvolvimento. Impulsionada por avanços em arquiteturas de redes neurais como o *Transformer* [31] e por modelos de linguagem de larga escala (LLMs) treinados em vastos conjuntos de dados de código [3], a GenAI demonstra capacidade para automatizar tarefas, auxiliar na tomada de decisão e aumentar a produtividade dos desenvolvedores [7].

As aplicações da GenAI em Engenharia de Software abrangem desde as fases iniciais de elicitação e análise de requisitos até as etapas de implementação, teste e manutenção. Na Engenharia de Requisitos, a GenAI pode auxiliar na automação da elicitação, análise e classificação de requisitos, facilitando a comunicação entre stakeholders e a formalização das necessidades do software [10]. No design de software, ferramentas como ChatGPT podem sugerir arquiteturas de sistema, identificar padrões de design apropriados e auxiliar na criação de diagramas e modelos, agilizando o processo de modelagem e prototipação [19].

Durante a implementação, a GenAI oferece suporte na geração, conclusão e sumarização de código, reduzindo o esforço manual e acelerando o desenvolvimento. Ferramentas como GitHub Copilot, baseadas em LLMs, demonstram capacidade para gerar código funcional a partir de descrições em linguagem natural [6, 13], completar trechos de código de forma inteligente [29] e auxiliar na refatoração [16] e documentação [1, 22]. Na fase de testes, a GenAI pode automatizar a geração de casos de teste, previsão de falhas e análise de cobertura de código, contribuindo para a garantia da qualidade do software [15, 32, 35]. Finalmente, na manutenção, a GenAI pode auxiliar na tradução de linguagens legadas para modernas, identificação de bugs e geração de *patches*, facilitando a evolução e manutenção de sistemas legados [33]. Apesar de seus benefícios promissores, a GenAI em Engenharia de Software ainda enfrenta desafios como a necessidade de dados específicos para o ajuste fino dos modelos, a garantia da segurança e qualidade do código gerado, a compreensão semântica do código e a integração eficiente em fluxos de trabalho existentes [19].

2.3 Métricas de Complexidade de Código

A mensuração da complexidade de código é uma prática fundamental na Engenharia de Software, pois permite uma avaliação quantitativa da dificuldade associada ao entendimento, teste e manutenção do software. As métricas de complexidade fornecem insights cruciais sobre a estrutura do código, ajudando a identificar áreas que podem ser propensas a erros ou que necessitem de refatoração para melhorar a qualidade e a manutenibilidade do software.

Dentre as diversas métricas existentes, destacam-se a Complexidade Ciclomática e a Complexidade Cognitiva. Essas métricas são reconhecidas e utilizadas tanto na indústria quanto na academia para avaliar a complexidade do código-fonte de maneira objetiva.

2.3.1 Complexidade Ciclomática. Proposta primeiramente no artigo *A Complexity Measure* — Uma Medida de Complexidade — por Thomas J. McCabe em 1978, numa época onde se enfrentava uma questão crítica dentro da Engenharia de Software: como modularizar um sistema de tal modo que o resultado seja ao mesmo tempo fácil de testar e de manter. Sendo assim, foi elaborada pelo autor a métrica de complexidade ciclomática onde, dado o grafo de fluxo de controle (*Control Flow Graph — CFG*) de um software, sua complexidade ciclomática (McCabe [17]) é dada por: $V(G) = E - N + 2P$, onde:

E = número de arestas do grafo; N = número de nós do grafo; P = número de componentes (subgrafos) conectados, em um CFG.

Quanto maior o valor de V(G), mais complexo é o código, e seu valor mínimo é de 1.

Como exemplo, temos que, para o grafo da Figura 1, a quantidade de arestas é 9, a de nós 8 e o número de componentes é 2. Logo, sua complexidade ciclomática será: $V(G) = 9 - 8 + 4 = 3$

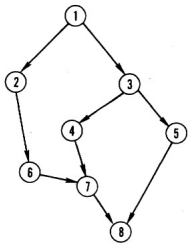

Figura 1: Exemplo de *Control Flow Graph — CFG*. Fonte: (McCabe [17])

2.3.2 Complexidade Cognitiva. É uma métrica proposta pela própria Sonar, diante de um cenário onde foram identificadas algumas limitações da Complexidade Ciclomática, a saber (Campbell [2]):

- Métodos com a mesma complexidade não possuem, necessariamente, a mesma dificuldade de manutenção;
- Por ter sido formulada no contexto do Fortran, ela não inclui estruturas modernas como *try/catch* e *lambdas*;
- Por fim, não é possível, à partir da Complexidade Ciclomática, saber se uma dada classe é extensa e fácil de manter ou pequena e complexa.

Tendo em vista este cenário, foi proposta a Complexidade Cognitiva, com o intuito de ser um valor que reflita de fato o grau de complexidade a nível de classe e de sistema. Ao invés do foco em um modelo matemático à partir de grafos, ela é avaliada buscando refletir o grau de esforço cognitivo necessário do programador para entender os fluxos avaliados (Campbell [2]).

Adicionalmente, na metodologia, são levadas em conta as peculiaridades de algumas linguagens, como ignorar *decorators* em *Python* e ignorar métodos puramente declarativos em *JavaScript*, por conta da adição recente — à época — de classes na linguagem e, por conseguinte, sua baixa adoção (Campbell [2]).

3 Metodologia

O objetivo deste estudo é investigar a capacidade de ferramentas de IA generativa em resolver problemas de programação. Para alcançar esse objetivo, foram definidas as seguintes questões de pesquisa:

- **QP1: Quão eficientes as ferramentas de IA Generativa são em resolver problemas de programação?**
  Com essa questão, buscamos avaliar a eficiência das ferramentas de IA generativa em resolver problemas de programação, o que é essencial para determinar se essas ferramentas podem realmente ajudar os programadores a aumentarem sua produtividade.

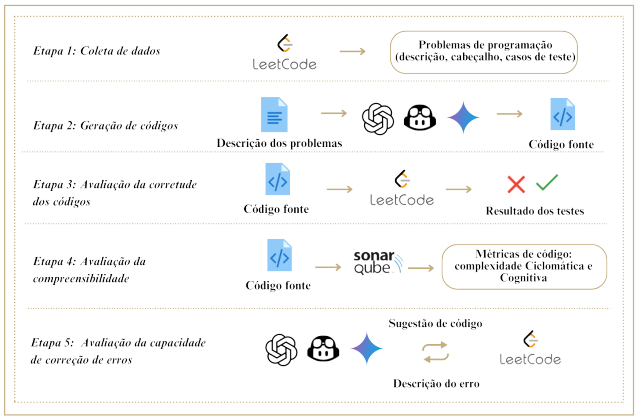

Figura 2: Esquema de etapas da configuração do estudo.

- **QP2: Quão compreensíveis são os códigos produzidos por estas ferramentas?**
  Nesta questão, examinamos a compreensibilidade dos códigos gerados, que é vital para garantir que os programadores possam entender, manter e evoluir o código produzido pelas ferramentas de IA. A compreensibilidade está diretamente ligada à qualidade e usabilidade do código.

- **QP3: Com que facilidade tais ferramentas são capazes de reparar os erros identificados no código gerado?**
  Aqui, investigamos a capacidade dessas ferramentas em reparar seus próprios erros, sendo um aspecto importante da robustez e confiabilidade das soluções geradas. Esta questão de pesquisa é particularmente relevante para contextos profissionais, onde a correção rápida e precisa de erros é fundamental.

Buscando responder às questões de pesquisa delineadas, este estudo se baseou em uma metodologia estruturada em cinco etapas, descritas na Figura 2, para avaliar as sugestões de código geradas pelos modelos de linguagem de Inteligência Artificial (IA) Copilot, ChatGPT e Gemini.

Etapa 1: Coleta de dados das questões. Essa etapa consiste em selecionar, preparar e organizar um conjunto de questões de programação do LeetCode[1] para análise. O objetivo é preparar as descrições dos problemas para a obtenção das sugestões de códigos das ferramentas. Dessa forma, tendo como base o estudo realizado por Nguyen e Nadi [18], foram selecionadas as mesmas 33 questões do LeetCode, que foram escolhidas por apresentarem uma variedade de dificuldades, sendo categorizadas como: 4 questões de nível fácil, 17 de nível médio e 12 de nível difícil.

Para o GitHub Copilot, foi realizado um processo manual de preparação dos dados das questões. As descrições completas de cada questão do LeetCode foram copiadas e colocadas como comentários nos arquivos de código-fonte criados para cada uma das linguagens do estudo. Este procedimento foi necessário para garantir que o Copilot pudesse acessar e compreender o contexto das questões ao gerar sugestões de código. Já para as ferramentas ChatGPT e

[1] https://leetcode.com/

Gemini, as descrições dos problemas foram inseridas como *prompts*. Essa abordagem permitiu que as ferramentas processassem o texto do prompt e gerassem sugestões de código com base nas descrições.

Etapa 2: Obtenção de sugestões do Copilot, ChatGPT e Gemini. Aqui, é feita a geração de sugestões de código para as questões obtidas na etapa anterior. O objetivo é coletar as respostas geradas para análise e comparação. Para o GitHub Copilot, os arquivos de código da Etapa 1 foram incluídos em um projeto no VSCode, onde ela foi acionada através de um plugin instalado na IDE. Posteriormente, coletamos sempre a primeira sugestão gerada pela IA e salvamos no mesmo arquivo. Para o ChatGPT e Gemini, foram criados prompts detalhados para cada questão, contendo as informações fornecidas pelo LeetCode. Essas ferramentas geraram códigos com base nos prompts, que foram então armazenados para posterior análise. O resultado foi um conjunto de arquivos de código fonte contendo as sugestões geradas por cada ferramenta para cada questão.

Etapa 3: Avaliação da corretude dos códigos. Esta etapa é dedicada a verificar a precisão das soluções de código geradas pelas ferramentas GitHub Copilot, ChatGPT e Gemini na etapa anterior. O objetivo é determinar se as sugestões fornecidas são corretas e funcionais. Para isso, cada uma das 396 soluções sugeridas foi manualmente inserida no ambiente de codificação do LeetCode, onde foram submetidas aos casos de teste associados a cada questão. O processo de verificação incluiu a execução dos códigos sugeridos para validar se eles passavam ou falhavam nos casos de teste. A saída dessa execução, com resultados de sucesso ou falha, foi registrada cuidadosamente. Tais registros permitem identificar se as soluções são totalmente corretas, parcialmente corretas (passando alguns testes e falhando em outros) ou incorretas (falhando em todos os testes).

O resultado dessa etapa foi um relatório detalhado que incluiu a lista de soluções que passaram ou falharam nos testes, a descrição de quaisquer mensagens de erro ou falhas ocorridas durante a execução dos testes e a porcentagem de casos de teste aprovados para cada solução, proporcionando uma visão quantitativa da eficácia de cada ferramenta em resolver os problemas propostos. Com essa análise, é possível avaliar comparativamente a capacidade das ferramentas de gerar soluções corretas e funcionais, identificando possíveis áreas de melhoria e pontos fortes de cada ferramenta.

Etapa 4: Avaliação da compreensibilidade. Esta etapa visa analisar a compreensibilidade do código gerado pelas ferramentas de inteligência artificial GitHub Copilot, ChatGPT e Gemini. O objetivo é avaliar a clareza e a simplicidade das soluções de código fornecidas.

Para essa avaliação, utilizamos a ferramenta de análise estática SonarQube[2], a qual calculou as métricas de complexidade nos arquivos fonte gerados em Java, JavaScript e Python. A versão utilizada foi escolhida devido à sua popularidade e à capacidade de identificar automaticamente problemas de código relacionados à legibilidade e manutenibilidade, como complexidade ciclomática, densidade de comentários, e outros indicadores de qualidade de código.

Devido ao fato da versão do SonarQube utilizada não suportar a linguagem C para o cálculo de métricas de complexidade ciclomática e de linhas de código, desenvolvemos um programa personalizado para calculá-las, o que permitiu uma análise consistente e comparável das soluções em C, alinhada às métricas aplicadas às outras linguagens.

O resultado dessa etapa foi um conjunto de pontuações de compreensibilidade para cada uma das 396 soluções geradas. Essas pontuações refletem aspectos como a simplicidade do código, a clareza dos comentários, e a facilidade de manutenção. A análise comparativa das métricas de complexidade permitiu identificar quais ferramentas geram código mais legível e fácil de entender, proporcionando insights sobre a usabilidade prática das soluções propostas pelas ferramentas de IA.

Etapa 5: Avaliação da capacidade de correção de erros: Nessa etapa, o foco é avaliar a habilidade das ferramentas GitHub Copilot, ChatGPT e Gemini em corrigir erros presentes nas soluções de código que não foram aceitas pelo LeetCode. O objetivo é entender como cada ferramenta lida com a depuração de código e a resolução de problemas identificados.

Para conduzir essa avaliação, copiamos a saída de erro do LeetCode para cada solução que não foi aceita, juntamente com o código sugerido inicialmente e o cabeçalho da questão. Esses elementos foram apresentados às ferramentas para que elas pudessem avaliar e corrigir seus próprios erros. O processo de depuração foi estruturado como uma série de interações, onde cada tentativa de reparo baseava-se nas respostas e diagnósticos anteriores fornecidos pelas ferramentas.

Para todas as ferramentas analisadas, utilizamos um "prompt padrão" como ponto de partida para o processo de correção. Nele, estavam inclusas a especificação da linguagem de programação, a descrição do erro retornado pelo LeetCode e a assinatura da função. No caso do GitHub Copilot, utilizamos sua interface de chat disponível no *gitHub.com*, que permite a interação com a ferramenta para perguntas e respostas relacionadas à codificação, permitindo que o Copilot, assim como as outras ferramentas, analisasse o problema e sugerisse correções. O "prompt padrão" segue a especificação abaixo:

```
<codigo atual>
"Check the "<Linguagem>"solution error and fix it:"
<output do LeetCode>
"Reply in <Linguagem>"
<cabecalho da funcao>
```

Listing 1: Modelo do prompt padrão.

O processo assemelhou-se a uma interação dialógica, a qual continuava enquanto as ferramentas mostravam progresso na correção dos erros. Se nenhuma melhoria significativa fosse observada, ele era interrompido. Para cada problema identificado, foi permitido um máximo de sete interações (ou prompts) para tentar corrigir o erro. O resultado dessa etapa incluiu um esboço das questões reparadas e não reparadas por linguagem e ferramenta, bem como um registro detalhado das interações de depuração, documentando a quantidade das tentativas de correção feita pelas ferramentas.

Os dados coletados e utilizados neste estudo, incluindo as sugestões de código geradas pelas ferramentas, os resultados dos testes de corretude e as análises de complexidade, estão disponíveis publicamente para fins de replicação e futuras pesquisas através do seguinte link: https://zenodo.org/records/13885245 [3].

[2]https://www.sonarsource.com/products/sonarqube/

[3]https://zenodo.org/records/13885245

	Questão/ Número da questão no LeetCode	Testes	Número (%) de casos de testes passados											
			CHATGPT				COPILOT				GEMINI			
			Python	Java	JavaScript	C	Python	Java	JavaScript	C	Python	Java	JavaScript	C
Fácil	Q1 (1)	60	60 (100%)	60 (100%)	60 (100%)	9 (15%)	60 (100%)	60 (100%)	60 (100%)	51 (85%)	63(100%)	63(100%)	63(100%)	0(0%)
	Q2 (118)	30	30 (100%)	30 (100%)	30 (100%)	30 (100%)	30 (100%)	30 (100%)	30 (100%)	30 (100%)	30(100%)	30(100%)	30(100%)	30(100%)
	Q3 (119)	34	34 (100%)	34 (100%)	34 (100%)	0 (0%)	34 (100%)	34 (100%)	34 (100%)	0 (0%)	34(100%)	21 (61,8%)	34(100%)	0(0%)
	Q4 (338)	15	15 (100%)	15 (100%)	15 (100%)	15 (100%)	15 (100%)	15 (100%)	0 (0%)	15 (100%)	0(0%)	15(100%)	15(100%)	0(0%)
Médio	Q5 (467)	82	8 (9,8%)	8 (9,8%)	0 (0%)	0 (0%)	13 (15,9%)	82 (100%)	82 (100%)	82 (100%)	82(100%)	82(100%)	82(100%)	82(100%)
	Q6 (264)	596	596 (100%)	596 (100%)	596 (100%)	0 (0%)	596 (100%)	596 (100%)	596 (100%)	0 (0%)	596(100%)	596(100%)	596(100%)	596(100%)
	Q7 (313)	87	87 (100%)	9 (10,3%)	0 (0%)	84 (96,6%)	87 (100%)	85 (97,7%)	87 (100%)	0 (0%)	87(100%)	85(97,7%)	87(100%)	84(96,6%)
	Q8 (396)	58	58 (100%)	58 (100%)	58 (100%)	0 (0%)	58 (100%)	58 (100%)	58 (100%)	58 (100%)	58(100%)	58(100%)	58(100%)	58(100%)
	Q9 (416)	141	141 (100%)	141 (100%)	141 (100%)	0 (0%)	141 (100%)	141 (100%)	141 (100%)	141 (100%)	141(100%)	141(100%)	141(100%)	0(0%)
	Q10 (435)	58	58 (100%)	58 (100%)	58 (100%)	11 (19%)	58 (100%)	58 (100%)	58 (100%)	11 (19%)	59(100%)	59(100%)	59(100%)	0(0%)
	Q11 (300)	55	55 (100%)	55 (100%)	55 (100%)	55 (100%)	55 (100%)	55 (100%)	55 (100%)	55 (100%)	55(100%)	55(100%)	9(16,4%)	55(100%)
	Q12 (368)	49	49 (100%)	49 (100%)	49 (100%)	49 (100%)	21 (42,9%)	49 (100%)	49 (100%)	0 (0%)	49(100%)	49(100%)	49(100%)	49(100%)
	Q13 (397)	47	47 (100%)	46 (97,9%)	47 (100%)	46 (97,9%)	47 (100%)	47 (100%)	47 (100%)	46 (97,9%)	47(100%)	46(97,9%)	47(100%)	1(2,1%)
	Q14 (343)	50	50 (100%)	50 (100%)	50 (100%)	5 (10%)	50 (100%)	50 (100%)	50 (100%)	50 (100%)	50(100%)	50(100%)	50(100%)	50(100%)
	Q15 (106)	202	202(100%)	202(100%)	202(100%)	0 (0%)	202(100%)	202(100%)	202(100%)	202(100%)	202(100%)	202(100%)	202(100%)	202(100%)
	Q16 (377)	15	15 (100%)	15 (100%)	15 (100%)	14 (93,3%)	15 (100%)	15 (100%)	15 (100%)	0 (0%)	16(100%)	16(100%)	16(100%)	14(87,5%)
	Q17 (322)	189	189 (100%)	189 (100%)	189 (100%)	189 (100%)	189 (100%)	189 (100%)	189 (100%)	189 (100%)	189(100%)	189(100%)	189(100%)	189(100%)
	Q18 (357)	9	9 (100%)	9 (100%)	9 (100%)	9 (100%)	3 (33,3%)	9 (100%)	8 (88,9%)	9 (100%)	9(100%)	9(100%)	9(100%)	9(100%)
	Q19 (464)	58	58 (100%)	58 (100%)	58 (100%)	0 (0%)	58 (100%)	58 (100%)	0 (0%)	0 (0%)	0(0%)	58(100%)	58(100%)	0(0%)
	Q20 (309)	210	210(100%)	210(100%)	210(100%)	0 (0%)	210(100%)	210(100%)	210(100%)	110(52,4%)	210(100%)	210(100%)	210(100%)	210(100%)
	Q21 (413)	15	15 (100%)	15 (100%)	15 (100%)	15 (100%)	15 (100%)	15 (100%)	15 (100%)	10 (66,7%)	15(100%)	15(100%)	15(100%)	15(100%)
Difícil	Q22 (354)	87	85 (97,7%)	85 (97,7%)	85 (97,7%)	85 (97,7%)	85 (97,7%)	85 (97,7%)	85 (97,7%)	0 (0%)	85(97,7%)	84(96,6%)	84(96,6%)	85(97,7%)
	Q23 (458)	18	17 (94,4%)	17 (94,4%)	17 (94,4%)	17 (94,4%)	17 (94,4%)	18 (100%)	17 (94,4%)	0 (0%)	26(100%)	26(100%)	26(100%)	26(100%)
	Q24 (115)	65	65 (100%)	65 (100%)	65 (100%)	0 (0%)	65 (100%)	65 (100%)	65 (100%)	60 (92,3%)	65(100%)	65(100%)	65(100%)	65(100%)
	Q25 (312)	73	73 (100%)	73 (100%)	73 (100%)	73 (100%)	73 (100%)	0 (0%)	73 (100%)	73 (100%)	0(0%)	0(0%)	0(0%)	0(0%)
	Q26 (410)	31	31 (100%)	31 (100%)	31 (100%)	31 (100%)	31 (100%)	31 (100%)	0 (0%)	0 (0%)	32(100%)	32(100%)	32(100%)	0(0%)
	Q27 (233)	38	38 (100%)	38 (100%)	38 (100%)	38 (100%)	34 (89,5%)	38 (100%)	35 (92,1%)	0 (0%)	6(15,8%)	6(15,8%)	6(15,8%)	6(15,8%)
	Q28 (363)	36	0 (0%)	36 (100%)	0 (0%)	0 (0%)	36 (100%)	36 (100%)	0 (0%)	0 (0%)	19(52,8%)	36(100%)	36(100%)	0(0%)
	Q29 (329)	138	138(100%)	138(100%)	138(100%)	0 (0%)	138(100%)	138(100%)	0 (0%)	0 (0%)	0(0%)	138(100%)	138(100%)	0(0%)
	Q30 (403)	54	54 (100%)	54 (100%)	54 (100%)	0 (0%)	54 (100%)	54 (100%)	0 (0%)	54 (100%)	55(100%)	25(45,5%)	0(0%)	0(0%)
	Q31 (466)	49	25 (51%)	40 (81,6%)	49 (100%)	25 (5%)	7 (14,3%)	40 (81,6%)	49 (100%)	49 (100%)	0(0%)	35(71,4%)	29(59,2%)	30(61,2%)
	Q32 (472)	43	27 (62,8%)	27 (62,8%)	27 (62,8%)	0 (0%)	42 (97,7%)	43 (100%)	0 (0%)	27 (62,8%)	0(0%)	0(0%)	0(0%)	0(0%)
	Q33 (446)	101	10 (9,9%)	8 (7,9%)	10 (9,9%)	4 (4%)	10 (9,9%)	101(100%)	101(100%)	4 (4%)	0(0%)	10(9,9%)	10(9,9%)	4(4%)
Aceita			26 (78,8%)	26 (78,8%)	26 (78,8%)	10 (30,3%)	24 (72,7%)	29 (87,9%)	22 (66,7%)	13 (39,4%)	23 (69,7%)	23 (69,7%)	25 (75,7%)	14 (42,4%)
Resposta Errada			4 (12,1%)	4 (12,1%)	4 (12,1%)	6 (18,2%)	6 (18,2%)	1 (3%)	1 (3%)	7 (21,2%)	4 (12,1%)	8 (24,2%)	8 (24,2%)	7 (21,2%)
Tempo Limite Excedido			2 (6,1%)	2 (6,1%)	1 (3%)	2 (6,1%)	3 (9,1%)	2 (6,1%)	3 (9,1%)	0 (0%)	1 (3%)	1 (3%)	0 (0%)	1 (3%)
Erro de Compilação			0 (0%)	0 (0%)	0 (0%)	9 (27,3%)	0 (0%)	0 (0%)	0 (0%)	11 (33,3%)	0 (0%)	1 (3%)	0 (0%)	6 (18,2%)
Erro de Tempo de Execução			1 (3%)	1 (3%)	2 (6,1%)	6 (18,2%)	0 (0%)	1 (3%)	7 (21,2%)	2 (6,1%)	5 (15,2%)	0 (0%)	0 (0%)	5 (15,2%)
Parcialmente Corretas			6 (18,2%)	8 (24,2%)	4 (12,1%)	10 (30,3%)	9 (27,3%)	3 (9,1%)	4 (12,1%)	8 (24,2%)	3 (9,1%)	8 (24,2%)	5 (15,2%)	7 (21,2%)

Figura 3: Correção das sugestões das ferramentas ChatGPT, Copilot e Gemini, respectivamente, para cada questão do LeetCode e status das submissões.

4 Resultados

Essa seção apresenta os resultados conforme direcionamento dado pelas questões de pesquisa apresentadas na Seção 3.

***RQ1:* Quão eficientes as ferramentas de IA Generativa são em resolver problemas de programação?**

Para responder a esse questionamento, foi investigada a capacidade das ferramentas em prover respostas corretas aos problemas extraídos do LeetCode. As tabelas apresentadas na Figura 3 fornecem uma visão abrangente da correção das sugestões de código geradas pelas ferramentas ChatGPT, Copilot e Gemini, conforme testadas em questões do LeetCode. Cada tabela está dividida em colunas para as diferentes linguagens de programação (Python, Java, JavaScript, C) e em linhas que representam as questões categorizadas como fáceis, médias e difíceis. As células dentro das tabelas mostram o quantidade e a porcentagem de casos de teste passados para cada questão, com uma codificação por cores para facilitar a visualização dos resultados: verde para questões que foram completamente aceitas; laranja para questões com respostas incorretas; cinza para questões que excederam o limite de tempo; vermelho para questões com erros de compilação; e amarelo para questões com erros em tempo de execução.

A taxa de questões 100% corretas variou substancialmente entre as linguagens de programação. Em Java, o Copilot destacou-se com 87,88% de questões corretas, seguido pelo ChatGPT com 78,79% e o Gemini com 69,70%. Em Python, o ChatGPT teve 78,79% de questões corretas, enquanto o Copilot apresentou 72,73% e o Gemini 69,70%. Em JavaScript, o ChatGPT alcançou 78,79% de questões corretas, seguido pelo Gemini com 75,76% e o Copilot com 66,67%. Já em C, os resultados foram mais baixos: o Gemini teve 42,42%, o Copilot 39,39% e o ChatGPT 30,30%.

As questões parcialmente corretas (*resposta errada* ou *tempo limite excedido*) também apresentaram variações significativas para as diferentes ferramentas e linguagens de programação. Em C, o ChatGPT apresentou 30,30% de questões parcialmente corretas, seguido pelo Copilot com 24,24% e o Gemini com 21,21%. Em Java, tanto o ChatGPT quanto o Gemini tiveram 24,24%, enquanto o Copilot apresentou 9,09%. Em JavaScript, o Gemini teve 15,15%, enquanto o Copilot e o ChatGPT tiveram 12,12%. Em Python, o Copilot teve 27,27%, o ChatGPT 18,18% e o Gemini 9,09%.

Os erros que impediram a execução dos testes (parcial ou completa) incluíram tempo limite excedido, erros de compilação e erros de tempo de execução. Em C, os erros de compilação foram predominantes, com 33,33% no Copilot e 27,27% no ChatGPT. Em termos de erros de tempo de execução, JavaScript teve 21,21% no Copilot, enquanto C e Java apresentaram 6,06% e 3,03%, respectivamente. Python e JavaScript tiveram 9,09% de erros de tempo limite excedido no Copilot, enquanto Java apresentou 6,06%. Para estes casos de erro por tempo limite excedido, os códigos sugeridos pelas ferramentas foram capazes de passar na maioria dos testes (acima de 80% dos casos de testes) em 17 dos 19 casos. Por fim, vale ressaltar que para a linguagem Python, o Gemini produziu códigos sem erros que impediram a execução dos testes (compilação, execução ou limite de tempo) para todas as questões.

Resumo da RQ1

As três ferramentas tiveram bons desempenhos para as linguagens Python, Java e JavaScript, com taxas de acerto (100% das questões) acima de 69.7%. O Gemini se destaca em Java (87.88%) e o ChatGPT pelo bom desempenho geral nas três linguagens (78.79%). Entretanto, na linguagem C o desempenho foi baixo no geral, se destacando o Gemini com 42.42% de respostas aceitas.

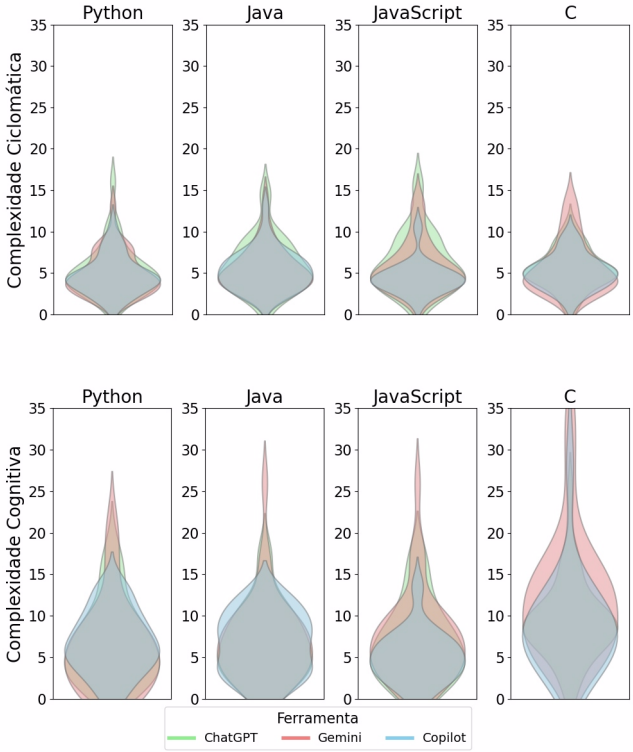

Figura 4: Distribuição dos valores de complexidade ciclomática e cognitiva.

***RQ2:* O quão compreensíveis são os códigos produzidos por estas ferramentas?**

A complexidade cognitiva e ciclomática dos códigos gerados pelas ferramentas foi computada conforme descrito na Seção 3 e os resultados são apresentados na Figura 4. Gráficos de violino foram utilizados para exibir as distribuições dos valores de complexidade ciclomática (superior) e cognitiva (inferior) para cada ferramenta e linguagem de programação. Adicionalmente, a Tabela 1 apresenta os resultados do teste estatístico de Mann-Whitney e do cálculo do Cliffs-Delta [12] para a comparação das distribuições de complexidade dos códigos gerados pelas diferentes ferramentas, detalhando os valores de *U-Statistic*, *p-value* e tamanho do efeito para cada par de ferramentas e tipo de complexidade.

Em Python, a complexidade ciclomática variou de 1 a 16, com o Copilot tendo a maioria das questões entre 2 e 7, o ChatGPT com

Tabela 1: Teste Estatístico de Mann-Whitney e Cliffs-Delta para Comparação das Distribuições de Complexidade.

Language	Tool1	Tool2	Cyclomatic			Cognitive		
			P-Value	Cliffs-Delta	Effect-Size	P-Value	Cliffs-Delta	Effect-Size
Python	Copilot	ChatGPT	0,706	-0,054	insignificante	0,571	0,082	insignificante
Python	Copilot	Gemini	0,927	0,014	insignificante	0,267	0,159	pequeno
Python	ChatGPT	Gemini	0,683	0,059	insignificante	0,562	0,084	insignificante
Java	Copilot	ChatGPT	0,539	-0,088	insignificante	0,877	0,023	insignificante
Java	Copilot	Gemini	0,959	0,008	insignificante	0,913	0,017	insignificante
Java	ChatGPT	Gemini	0,509	0,095	insignificante	0,949	-0,010	insignificante
JavaScript	**Copilot**	**ChatGPT**	**0,037**	**-0,295**	**pequeno**	0,184	-0,190	pequeno
JavaScript	**Copilot**	**Gemini**	0,087	-0,242	pequeno	**0,073**	**-0,256**	**pequeno**
JavaScript	ChatGPT	Gemini	0,609	0,073	insignificante	0,738	-0,049	insignificante
C	Copilot	ChatGPT	0,520	-0,092	insignificante	0,499	-0,097	insignificante
C	**Copilot**	**Gemini**	0,871	-0,024	insignificante	**0,040**	**-0,295**	**pequeno**
C	ChatGPT	Gemini	0,641	0,067	insignificante	0,157	-0,203	pequeno

várias questões entre 4 e 10, e o Gemini entre 3 e 8. A complexidade cognitiva variou de 0 a 19, com o Copilot apresentando 70% das questões abaixo de 10, o ChatGPT 80% das questões acima de 5 e o Gemini mostrando uma faixa de 0 a 22.

Em Java, a complexidade ciclomática variou de 1 a 15, com o Copilot e o ChatGPT apresentando a maioria das questões entre 4 e 9, enquanto o Gemini variou de 2 a 14. A complexidade cognitiva variou de 0 a 26. O Copilot teve 75% das questões abaixo de 10, o ChatGPT 70% das questões acima de 5, e o Gemini apresentando várias questões acima de 10.

Para JavaScript, a complexidade ciclomática variou de 1 a 16, com o Copilot tendo a maioria das questões entre 2 e 6, o ChatGPT entre 3 e 10, e o Gemini entre 4 e 9. A complexidade cognitiva variou de 0 a 26, com o Copilot apresentando a maioria das questões abaixo de 10, o ChatGPT com várias questões acima de 10, e o Gemini mostrando uma ampla faixa.

Para a linguagem C, a complexidade ciclomática para todas as ferramentas variou de 1 a 16, com o Copilot e o ChatGPT concentrando a maioria das questões entre 3 e 10, enquanto o Gemini teve uma distribuição mais ampla dos valores de complexidade dos códigos gerados. O Copilot apresentou uma distribuição equilibrada de complexidade cognitiva, com 70% das questões abaixo de 10. O ChatGPT também apresentou uma distribuição ampla, com 66,7% das questões abaixo de 10, mas com algumas questões atingindo até 25. Já o Gemini mostrou uma maior variabilidade, com várias questões de alta complexidade, alcançando até 35.

Por fim, a análise visual dos gráficos de violino na Figura 4 sugere que o Copilot apresenta, em geral, menor complexidade de código em comparação ao ChatGPT e Gemini, com distribuições de valores concentradas em níveis mais baixos. No entanto, os resultados apresentados na Tabela 1 indicam que essa tendência nem sempre se traduz em diferenças estatisticamente significativas. Na maioria dos casos, não há uma diferença significativa no desempenho geral entre as ferramentas. Apenas em cenários específicos, especialmente para as linguagens JavaScript e C, foram encontradas diferenças estatisticamente significativas ($p\text{-}value < 0.05$). Especificamente, para JavaScript, o Copilot apresentou complexidade ligeiramente inferior em relação ao ChatGPT e Gemini, tanto na análise de complexidade ciclomática quanto na de complexidade cognitiva. Resultados similares foram observados para a linguagem C, na análise de complexidade cognitiva, com o Copilot demonstrando menor complexidade em relação ao Gemini, assim como o ChatGPT em relação ao Gemini. Em ambos os casos, adotando intervalos de Romano et al. [27], a diferença entre as distribuições é pequena ($|d| < 0.33$).

Resumo da RQ2
A análise da complexidade do código gerado por Copilot, ChatGPT e Gemini sugere que o Copilot tende a produzir código menos complexo, embora essa diferença nem sempre seja estatisticamente significativa. Diferenças significativas, porém pequenas, foram observadas principalmente em JavaScript e C, favorecendo o Copilot.

***RQ3:* Com que facilidade tais ferramentas são capazes de reparar os erros identificados no código gerado?**

A capacidade de reparo (autocorreção) das ferramentas foi avaliada em um processo interativo e iterativo, no qual as ferramentas recebiam o feedback sobre os erros identificados pelo LeetCode e tinham a oportunidade de corrigi-los. Esse processo de interação com as ferramentas se repetia até que uma solução correta fosse obtida (sucesso do reparo) ou quando após diversas tentativas não era observado evolução nas novas soluções apresentadas (insucesso). As Figuras 5, 6 e 7 apresentam os resultados desse experimento para as três ferramentas (Copilot, ChatGPT e Gemini, respectivamente), discriminando o sucesso e fracasso no reparo por tipo de erro e linguagem de programação.

Analisando a capacidade de reparo de erros, observamos que o Copilot (Figura 5) destacou-se em C, conseguindo reparar 24 dos 25 erros identificados (96%), sendo: onze de onze erros de compilação, seis de seis erros de execução (*runtime*), um de um erro de limite de tempo excedido e seis de sete erros em testes (*wrong*. Em Java, dos quatro erros identificados, o Copilot foi capaz de corrigir três (75%), sendo dois erros de limite de tempo excedido e uma questão com

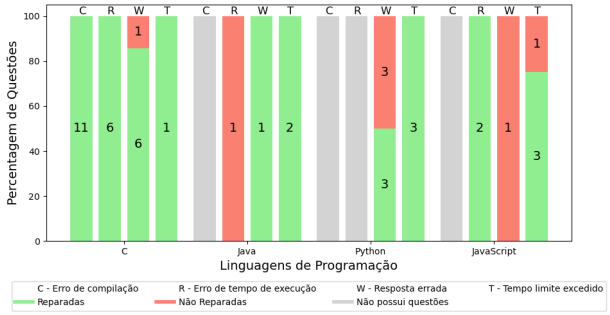

Figura 5: Copilot - Questões reparadas e não reparadas por tipo de erro e linguagem.

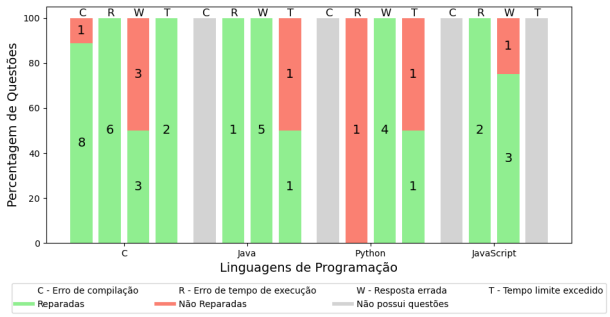

Figura 6: ChatGPT - Questões reparadas e não reparadas por tipo de erro e linguagem.

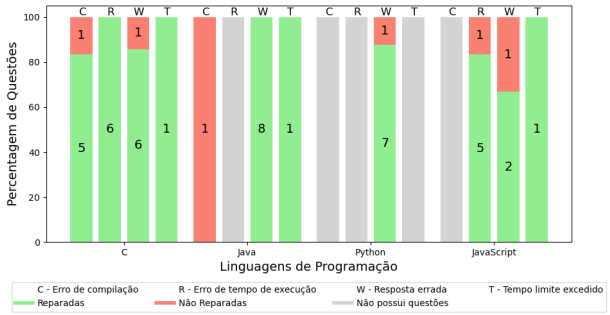

Figura 7: Gemini - Questões reparadas e não reparadas por tipo de erro e linguagem.

resposta incorreta. Apenas um erro de execução não foi corrigido. Em Python e JavaScript, embora bastante significativa, a capacidade de reparo foi menor, respectivamente 67% e 72%. No primeiro caso, foi capaz de reparar três de seis questões com respostas incorretas (50%) e todas as três questões com erros de tempo limite excedido. Em JavaScript, foram reparadas as duas questões com erros de execução, porém o Copilot não conseguiu reparar o código da questão com falha nos testes e um dos quatro erros de limite de tempo excedido (75%).

O ChatGPT (Figura 6) apresentou uma maior variabilidade na eficiência de reparação de erros. Em C, reparou oito de nove erros

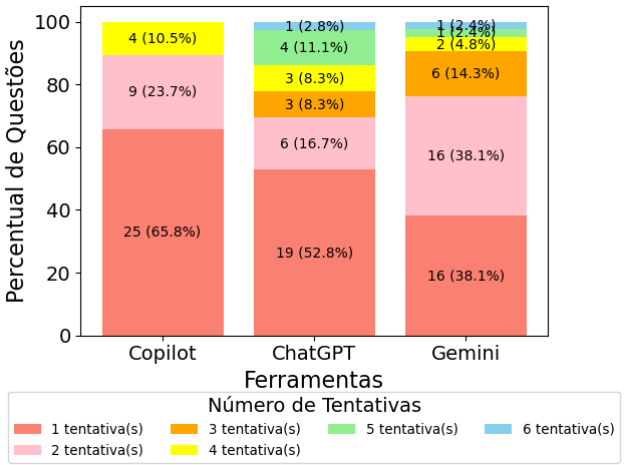

Figura 8: Número de tentativas para reparar erros nos códigos fontes sugeridos pelas ferramentas

de compilação (88,9%), todos os seis erros de execução e os dois erros de limite de tempo excedido. Entretanto, foi capaz de reparar apenas 50% das questões com erros nos testes. Em Java, reparou 100% das questões com respostas incorretas e com erro de execução, mas não conseguiu reparar um erro de tempo limite excedido. Em JavaScript, reparou 2 de 2 questões com erros de execução (100%) e todas as questões com respostas incorretas (100%). Em Python, reparou 4 de 5 questões com respostas incorretas (80%) e 1 de 2 questões com erros de tempo limite excedido (50%).

O Gemini (Figura 7) destacou-se em Java, reparando todas as 8 questões com respostas incorretas (100%) e 1 erro de tempo limite excedido(100%). Em C, reparou 5 de 6 erros de compilação (83,3%) e todos os 6 erros de execução (100%). Em Python, reparou 7 de 8 questões com respostas incorretas (87,5%) e 1 erro de tempo limite excedido (100%). Em JavaScript, reparou 5 de 6 questões com erros de execução (83,3%) e 1 erro de tempo limite excedido (100%).

Ao realizar o experimento para responder esta questão, percebemos que nem todos os problemas foram resolvidos no primeiro prompt. A persistência dos erros e a resposta da ferramenta determinaram o número de tentativas de reparo. Por exemplo, um erro de compilação como "***unknown type name TrieNode***" indicava que a definição da estrutura ***TrieNode*** estava ausente ou incorreta. Se a ferramenta não resolvesse esse erro em sucessivas interações, o problema era dado como não reparado. Isso deixou claro que a quantidade de prompts foi subjetiva e dependia do desempenho da ferramenta.

Adicionalmente, analisamos a quantidade de prompts necessários para cada ferramenta reparar suas respectivas respostas não aceitas pelo LeetCode. A Figura 8 detalha o número de tentativas (interações com a ferramenta) necessárias para o reparo. O Copilot se destacou como a ferramenta capaz de reparar mais rapidamente o problema, resolvendo 65,8% das questões na primeira tentativa e atingindo 100% dos reparos na quarta tentativa. O ChatGPT, teve uma eficiência inicial de 52,8%, atingindo 94,9% na sexta tentativa (com 8,3% resolvidos na quinta). Por fim, o Gemini

começou com 38,1%, aumentando para 79,6% na segunda tentativa e atingindo 100% na quinta.

Resumo da RQ3
A análise da capacidade de reparo de erros em códigos gerados pelas ferramentas revelou um desempenho geral positivo, embora com variações a depender da linguagem e do tipo de erro. Copilot obteve alta eficácia em C (96%), enquanto ChatGPT demonstrou maior variabilidade, com 71% de sucesso em Java e 67% em JavaScript e Python. Gemini apresentou melhor desempenho em Java (90%), seguido por Python (88%) e C (84%).

5 Discussões

Este estudo buscou aprofundar a compreensão sobre as ferramentas de geração de código conduzidas por IA, expandindo o escopo da pesquisa de Nguyen e Nadi [18]. Enquanto os autores focaram na avaliação do GitHub Copilot em termos de corretude e compreensibilidade, este estudo ampliou a análise para incluir o ChatGPT e o Gemini, além de introduzir a autocorreção como um novo eixo de avaliação. Além disso, foi utilizado o Copilot baseado em uma versão mais recente do modelo GPT: enquanto Nguyen e Nadi utilizaram a versão alimentada pelo GPT-3, nosso estudo utilizou a ferramenta alimentada pelo GPT-3.5 [23].

Comparando os resultados do presente estudo com os de Nguyen e Nadi, houve um grande aumento na taxa de acerto do Copilot em Java, saltando de 57% para 88%, o que sugere que o modelo passou por aprimoramentos importantes desde então. Vale ressaltar, entretanto, que essa evolução, a escolha dos conjuntos de problemas e as versões das bibliotecas utilizadas podem influenciar a reprodutibilidade dos resultados, demandando cautela nas comparações diretas.

Em relação à compreensibilidade do código gerado, ambos os estudos utilizaram as métricas de complexidade ciclomática e cognitiva, convergindo na observação de que as ferramentas, em geral, produzem código com baixa complexidade. Nguyen e Nadi relataram que o GitHub Copilot gerou código com complexidade cognitiva abaixo de 10 em 60% das questões analisadas, indicando uma tendência para gerar código mais simples. No entanto, este estudo aprofundou a análise e descobriu que o Copilot manteve a complexidade cognitiva abaixo de 10 em 70% das questões, sugerindo uma melhoria na simplicidade do código gerado. Além disso, a complexidade ciclomática média observada no estudo atual foi de 5,2, comparada a uma média de 6,5 no estudo de Nguyen e Nadi, apontando para um código estruturalmente menos complexo na geração mais recente. Nesse contexto, vale ressaltar que a baixa complexidade, embora desejável, não garante a qualidade do código em termos de legibilidade, manutenibilidade e aderência a boas práticas de programação.

A análise da capacidade de autocorreção, o diferencial em relação ao trabalho de Nguyen e Nadi, demonstra que as ferramentas apresentam um potencial em identificar e corrigir erros, com destaque para o Copilot em C e o Gemini em Java, sendo importante por espelhar o processo iterativo inerente ao desenvolvimento de software, no qual o desenvolvedor alterna entre codificação, manutenção (correção) e evolução do código. A interação com a IA, nesse contexto, aproxima-se da dinâmica natural do trabalho do desenvolvedor, permitindo que ele refine e direcione a geração de código por meio de um diálogo interativo de feedback e refinamento. A variabilidade do sucesso da autocorreção em função da linguagem de programação e da complexidade do erro, contudo, indica a necessidade de pesquisas adicionais para o desenvolvimento de mecanismos mais robustos de diagnóstico e solução de problemas.

É fundamental reconhecer que este estudo, assim como o de Nguyen e Nadi, baseou-se em um conjunto específico de problemas do LeetCode, o que pode não refletir a complexidade e a diversidade de cenários reais de desenvolvimento de software. Faz-se necessária a realização de estudos em contextos reais, com bases de código e requisitos mais complexos, para uma avaliação mais abrangente. Além disso, a interação entre desenvolvedores e ferramentas de IA merece atenção. A confiança na IA, a compreensão do código gerado, o tempo dedicado à revisão e adaptação e o impacto na experiência do desenvolvedor são aspectos relevantes a serem considerados em futuras pesquisas.

Os resultados deste estudo, em consonância com o trabalho de Nguyen e Nadi, reforçam a rápida evolução das ferramentas de geração de código conduzidas por IA e a importância de pesquisas contínuas para acompanhar seu desenvolvimento. As ferramentas demonstraram potencial para auxiliar desenvolvedores em tarefas de codificação, mas a supervisão e o julgamento humano continuam sendo importantes. A compreensão das capacidades e limitações dessas ferramentas permite seu uso mais eficaz, impulsionando a produtividade e a qualidade no desenvolvimento de software.

6 Ameaças a Validade

A seguir, serão apresentadas algumas ameaças à validade que, apesar de terem sido alvo de medidas mitigatórias, devem ser consideradas na interpretação dos resultados.

A validade interna pode ter sido comprometida pela variabilidade na dificuldade das questões do LeetCode. Para mitigar este problema, foram selecionadas questões com diferentes níveis de dificuldade (fácil, médio e difícil) e mantidas as mesmas questões do estudo de Nguyen e Nadi (2022), permitindo uma comparação direta dos resultados. A subjetividade na elaboração dos prompts para o ChatGPT e Gemini também pode ter introduzido viés. Para minimizar este problema, foram elaborados com base nas informações fornecidas pelo LeetCode, buscando manter a neutralidade.

Os resultados obtidos com o Copilot, ChatGPT e Gemini podem não ser generalizáveis para outras ferramentas de IA ou para outras linguagens de programação além das quatro estudadas (Python, Java, JavaScript e C). A escolha destas ferramentas foi baseada na sua relevância no contexto atual de geração de código por IA, enquanto as linguagens foram selecionadas por representarem diferentes paradigmas e por serem amplamente utilizadas, constando entre as 10 mais populares em 2023 [5]. O estudo também pode não refletir as complexidades do desenvolvimento de software em cenários reais, que envolvem a interação com diversas ferramentas e bibliotecas externas. Contudo, a utilização de um conjunto de dados reconhecido como o LeetCode fornece uma base sólida para a análise comparativa das ferramentas.

Com relação à construção dos experimentos, as métricas de compreensibilidade (complexidade ciclomática e cognitiva), apesar de não abrangerem todos os aspetos da compreensibilidade do código, foram escolhidas por serem amplamente utilizadas e fornecerem uma medida quantitativa da complexidade do código.

Adicionalmente, a definição de "reparo de erros" neste estudo se baseou na capacidade de compilar, executar e obter aprovação nos testes do LeetCode, desconsiderando a qualidade do código em termos de legibilidade, eficiência e manutenibilidade. Esta definição se justifica pela natureza do estudo, que visa avaliar a capacidade das ferramentas em gerar código funcional e pela dificuldade em estabelecer critérios universais para a qualidade do código.

Em resumo, é importante ter em mente estas limitações ao analisar os resultados da pesquisa. As medidas de mitigação adotadas visam fortalecer a validade do estudo, mas reconhecemos que pesquisas futuras devem se dedicar a explorar estas limitações com maior profundidade, utilizando diferentes conjuntos de dados, métricas e abordagens de análise.

7 Trabalhos Relacionados

Nguyen e Nadi [18] avaliaram a correção e a compreensibilidade de sugestões de código geradas pelo GitHub Copilot, em 33 questões do LeetCode, criando consultas em Python, Java, JavaScript e C, para o Copilot e, depois, executando os testes fornecidos pelo LeetCode e avaliando o código com o SonarQube. Java teve a maior pontuação de correção (57%) e todas as linguagens apresentaram boa compreensibilidade. Dentre as limitações, estavam o código que poderia ser simplificado e métodos auxiliares indefinidos.

Rao et al. [26] introduziram o LowCoder, uma ferramenta *low code* que combina programação visual (LowCoderVP) e programação em linguagem natural assistida por IA (LowCoderNL) para explorar como essas modalidades ajudariam os desenvolvedores a desenvolver código voltado para IA. A ferramenta se mostrou útil em casos nos quais os participantes tinham um objetivo claro, porém menos eficaz para novatos sem clareza sobre seus objetivos.

Corso et al. [4] apresentaram uma avaliação empírica do GitHub Copilot, Tabnine, ChatGPT e Google Bard na geração de métodos Java, utilizando um conjunto de 100 métodos obtidos de projetos open-source reais. A maioria dos métodos corretos foi gerada pelo Copilot e, junto com o Tabnine, foi a que melhor aderiu ao estilo de codificação dos desenvolvedores. Complexidade, eficiência e tamanho foram todos semelhantes em todos os assistentes.

Pearce et al. [24] investigam a segurança do código gerado pelo GitHub Copilot, focando na prevalência e nas condições que levam à geração de código inseguro. Eles projetaram 89 cenários relevantes para as Enumerações Comuns de Fraquezas (CWEs) de alto risco da lista "Top 25" da MITRE e pediram ao Copilot para completá-los. 40% do código gerado era vulnerável e alguns cenários e padrões levavam a uma prevalência maior de código inseguro. Concluíram que, embora o Copilot possa ser útil, seu código seria melhor utilizado juntamente com outras práticas e ferramentas de segurança.

Yetiştiren et al. [34] conduziram um estudo empírico para comparar o desempenho do GitHub Copilot, Amazon CodeWhisperer e ChatGPT, avaliando a validade, a correção, a segurança, a confiabilidade e a manutenibilidade do código, usando o dataset *HumanEval*, que contém problemas com entradas e saídas esperadas predefinidas. O ChatGPT foi o melhor em correção e validade do código, enquanto o Copilot foi o melhor em confiabilidade do código e o CodeWhisperer teve menos dívida técnica. Não houve diferença significativa entre os três em termos de segurança.

Vaithilingam et al. [30] avaliaram a usabilidade do GitHub Copilot, pedindo a 24 participantes que completassem tarefas de programação em Python usando o Copilot e o Intellisense. 19 dos 24 participantes preferiram o Copilot, mas não houve diferença significativa no tempo de conclusão das tarefas e no sucesso entre os dois. Os desafios encontrados pelos autores incluíam entender, editar e depurar o código gerado pelo Copilot, o que levou à preferência, em alguns casos, pelo Intellisense.

Nosso trabalho avança o conhecimento ao investigar não apenas a correção e compreensibilidade das soluções de código geradas por IA, como feito por Nguyen e Nadi [18], mas também ao avaliar a capacidade dessas ferramentas em reparar seus próprios erros.

Além disso, foram incluídas as análises do ChatGPT e Gemini, ferramentas que não foram abordadas por Nguyen e Nadi [18], ampliando assim o escopo da pesquisa. Diferente de Rao et al. [26], que focaram em ferramentas low-code, nosso estudo concentra-se em ferramentas de geração de código assistidas por IA para linguagens específicas. Enquanto Corso et al. [4] e Yetiştiren et al. [34] compararam várias ferramentas de IA, nosso estudo destaca a análise aprofundada das métricas de complexidade e compreensibilidade.

8 Conclusão

Este estudo investigou a corretude, compreensibilidade e capacidade de reparo de código gerado por três ferramentas populares de IA generativa: GitHub Copilot, ChatGPT e Gemini. Ao avaliar as soluções geradas para 33 questões do LeetCode em Java, JavaScript, Python e C, observou-se que o Copilot obteve o melhor desempenho em Java (87,88% de acurácia), enquanto o ChatGPT demonstrou consistência em Python, Java e JavaScript (78,79%). O Gemini se destacou em JavaScript, com 75,76% de acurácia. A linguagem C foi desafiadora para todas as ferramentas, com taxas de acerto inferiores a 50%. Em termos de compreensibilidade, o Copilot gerou código com menor complexidade cognitiva de forma mais consistente, enquanto ChatGPT e Gemini apresentaram maior variabilidade. A capacidade de autocorreção variou entre as ferramentas, com o Copilot demonstrando alta eficiência inicial, enquanto ChatGPT e Gemini melhoraram após algumas tentativas.

As descobertas deste estudo contribuem para a compreensão das capacidades e limitações das ferramentas de IA generativa para a geração de código, fornecendo insights para desenvolvedores e pesquisadores. Trabalhos futuros podem expandir a análise para um conjunto mais amplo de questões do LeetCode ou de outras plataformas, variando mais os domínios e níveis de complexidade. Outra possibilidade é aprofundar a investigação dos casos em que as ferramentas não conseguiram gerar soluções corretas, mesmo após múltiplas tentativas de reparo, analisando as causas dessas falhas e as características dos problemas enfrentados.

8.1 Agradecimentos

O presente trabalho foi realizado com apoio da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Código de Financiamento (403512/2021-2)

Referências

[1] T. Ahmed and P. Devanbu. 2023. Few-Shot Training LLMs for Project-Specific Code-Summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3551349.3559555

[2] G. Ann Campbell. 2023. *Cognitive Complexity - a new way of measuring understandability*. Technical Report. SonarSource S.A. https://www.sonarsource.com Version 1.7, 29 August 2023.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[4] Vincenzo Corso, Leonardo Mariani, Daniela Micucci, and Oliviero Riganelli. 2024. Generating Java Methods: An Empirical Assessment of Four AI-Based Code Assistants. *arXiv preprint arXiv:2402.08431v2* (2024). https://doi.org/10.48550/arXiv.2402.08431v2

[5] Kyle Daigle. 2024. Octoverse: The state of open source and rise of AI in 2023. https://github.blog/news-insights/research/the-state-of-open-source-and-ai/

[6] Armin Moradi Dakhel, Vahid Majdinasab, Ashkan Nikanjam, Foutse Khomh, Michael C. Desmarais, and Zhen Ming (Jack) Jiang. 2023. GitHub Copilot AI Pair Programmer: Asset or Liability? *Journal of Systems and Software* 203 (2023), 111734. https://doi.org/10.1016/j.jss.2023.111734

[7] Thomas Dohmke, Marco Iansiti, and Greg Richards. 2023. Sea change in software development: Economic and productivity analysis of the ai-powered developer lifecycle. *arXiv preprint arXiv:2306.15033* (2023).

[8] Christof Ebert and Panos Louridas. 2023. Generative AI for Software Practitioners. *IEEE Software* 40 (7 2023), 30–38. Issue 4. https://doi.org/10.1109/MS.2023.3265877

[9] Christof Ebert and Panos Louridas. 2023. Generative AI for software practitioners. *IEEE Software* 40, 4 (2023), 30–38.

[10] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh. 2023. AI-Based Question Answering Assistance for Analyzing Natural-Language Requirements. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1277–1289. https://doi.org/10.1109/ICSE48619.2023.00113

[11] Stefan Feuerriegel, Jochen Hartmann, Christian Janiesch, and Patrick Zschech. 2024. Generative ai. *Business & Information Systems Engineering* 66, 1 (2024), 111–126.

[12] Robert J. Grissom and John J. Kim. 2012. *Effect Sizes for Research: A Broad Practical Approach* (2 ed.). Routledge, New York, NY.

[13] S. Imai. 2022. Is GitHub Copilot a Substitute for Human Pair-Programming? An Empirical Study. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 319–321. https://doi.org/10.1145/3510454.3522684

[14] LeetCode. 2023. The World's Leading Online Programming Learning Platform. Disponível em: <https://leetcode.com/>. Acesso em 10 jun 2023.

[15] H. Liu, L. Liu, C. Yue, Y. Wang, and B. Deng. 2023. AutoTestGPT: A System for the Automated Generation of Software Test Cases Based on ChatGPT. Available at SSRN 4584792.

[16] A. Madaan, A. Shypula, U. Alon, M. Hashemi, P. Ranganathan, Y. Yang, G. Neubig, and A. Yazdanbakhsh. 2023. Learning Performance-Improving Code Edits. *arXiv:2302.07867* (2023).

[17] Thomas J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. https://doi.org/10.1109/TSE.1976.233837

[18] Nhan Nguyen and Sarah Nadi. 2022. An Empirical Evaluation of GitHub Copilot's Code Suggestions. In *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*. Pittsburgh, PA, USA.

[19] Anh Nguyen-Duc et al. 2023. Generative Artificial Intelligence for Software Engineering - A Research Agenda. *arXiv preprint arXiv:2310.18648* (2023).

[20] Anh Nguyen-Duc, Beatriz Cabrero-Daniel, Adam Przybylek, Chetan Arora, Dron Khanna, Tomas Herda, Usman Rafiq, Jorge Melegati, Eduardo Guerra, Kai-Kristian Kemell, et al. 2023. Generative Artificial Intelligence for Software Engineering–A Research Agenda. *arXiv preprint arXiv:2310.18648* (2023).

[21] Anh Nguyen-Duc, Beatriz Cabrero Daniel, Adam Przybylek, Chetan Arora, Dron Khanna, Tomas Herda, Usman Rafiq, Jorge Melegati, Eduardo Guerra, Kai-Kristian Kemell, Mika Saari, Zheying Zhang, Huy Le, Tho Quan, and Pekka Abrahamsson. 2023. Generative Artificial Intelligence for Software Engineering - A Research Agenda. *CoRR* abs/2310.18648 (2023). https://doi.org/10.48550/ARXIV.2310.18648 arXiv:2310.18648

[22] D. Noever and K. Williams. 2023. Chatbots as Fluent Polyglots: Revisiting Breakthrough Code Snippets. *arXiv:2301.03373 [cs]* (2023). https://doi.org/10.48550/arXiv.2301.03373

[23] OpenAI. 2024. OpenAI Codex. https://openai.com/index/openai-codex/. Accessed: 2024-08-02.

[24] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2021. An Empirical Cybersecurity Evaluation of GitHub Copilot's Code Contributions. *arXiv preprint arXiv:2108.09293* (2021). https://doi.org/10.48550/arxiv.2108.09293v2

[25] Luis Perez, Lizi Ottens, and Sudharshan Viswanathan. 2021. Automatic Code Generation using Pre-Trained Language Models. *California* 1 (feb 2021), 1–9.

[26] Nikitha Rao, Jason Tsay, Kiran Kate, Vincent J. Hellendoorn, and Martin Hirzel. 2023. AI for Low-Code for AI. *arXiv preprint arXiv:2305.20015* (2023). https://doi.org/10.48550/arxiv.2305.20015

[27] James Romano, Jeffrey D Kromrey, Jennifer Coraggio, and Judith Skowronek. 2006. Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?. In *Annual Meeting of the Florida Association of Institutional Research*. 1–3.

[28] Stuart Russell and Peter Norvig. 2022. *Inteligência Artificial: Uma Abordagem Moderna* (4 ed.). Pearson, São Paulo.

[29] J. Sun, Q. V. Liao, M. Muller, M. Agarwal, S. Houde, K. Talamadupula, and J. D. Weisz. 2022. Investigating Explainability of Generative AI for Code Through Scenario-Based Design. In *27th International Conference on Intelligent User Interfaces, IUI '22*. Association for Computing Machinery, 212–228. https://doi.org/10.1145/3490099.3511119

[30] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI EA '22: Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. USA, 1–7.

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. Attention is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. https://arxiv.org/abs/1706.03762

[32] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang. 2023. Software Testing with Large Language Model: Survey, Landscape, and Vision. *arXiv preprint arXiv:2307.07221* (2023).

[33] Justin D. Weisz, Michael Muller, Stephanie Houde, Justin Richards, Stephanie I. Ross, Fernando Martinez, Mayank Agarwal, and Kavitha Talamadupula. 2021. Perfection Not Required? Human-AI Partnerships in Code Translation. In *26th International Conference on Intelligent User Interfaces, IUI '21*. Association for Computing Machinery, 402–412. https://doi.org/10.1145/3397481.3450656

[34] Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778

[35] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *arXiv preprint arXiv:2305.04207* (2023).